

Aggressive FUZZing: From SSRF to Redis Password Spraying and Mass Reverse Shells

By J0o1ey · Translated from the Chinese original

0x00 Preface

Friends interested in technical exchange or penetration testing training are welcome to contact me via QQ/WeChat: 547006660.

If you need code auditing, penetration testing, or red-vs-blue team security services, feel free to reach out to my company.

We also run a 2,000-member security community group — group number 820783253. Experts are welcome to join the discussion.

I've hunted quite a few bug bounties lately, but most of them were hands-only bugs; ones that actually require thinking are rare.

Today I ran into an excellent case of the latter, hence this article.

0x01 Batch Directory FUZZing Uncovers a Hidden Interface

I had been hunting this big vendor for almost two weeks, and had been too busy with company work to continue recently.

But the whole time, ffuf with a wordlist had been running on my server, batch-scanning the vendor's assets for directories. Today I checked the scan results on the server — and it delivered.



The interface URL: <https://xxx.xxxx.com/xxxx/start>

Let's visit it directly.



The interface handed us a list of usable API links.

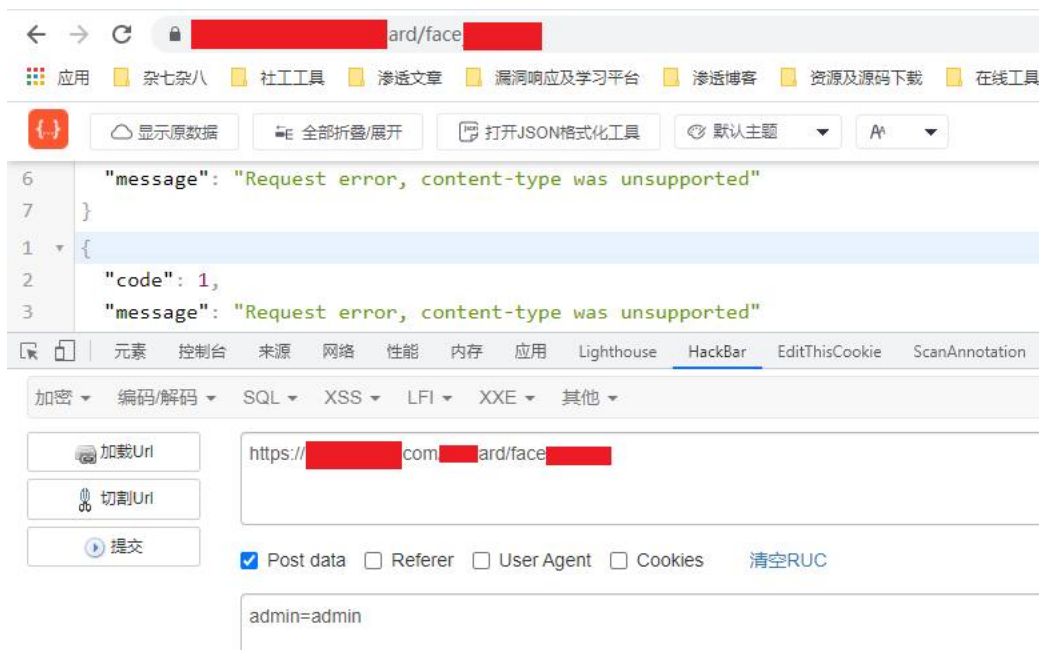
Testing the concatenated endpoints one by one, we found an interface named `face_xxxx` that looked promising.

0x02 FUZZing the Request Format and Parameters

Visiting the interface returned "Method Not Allowed", a 405. Clearly we needed to switch to POST.



POSTing some arbitrary parameters, the interface responded: "Request error, content-type was unsupported".



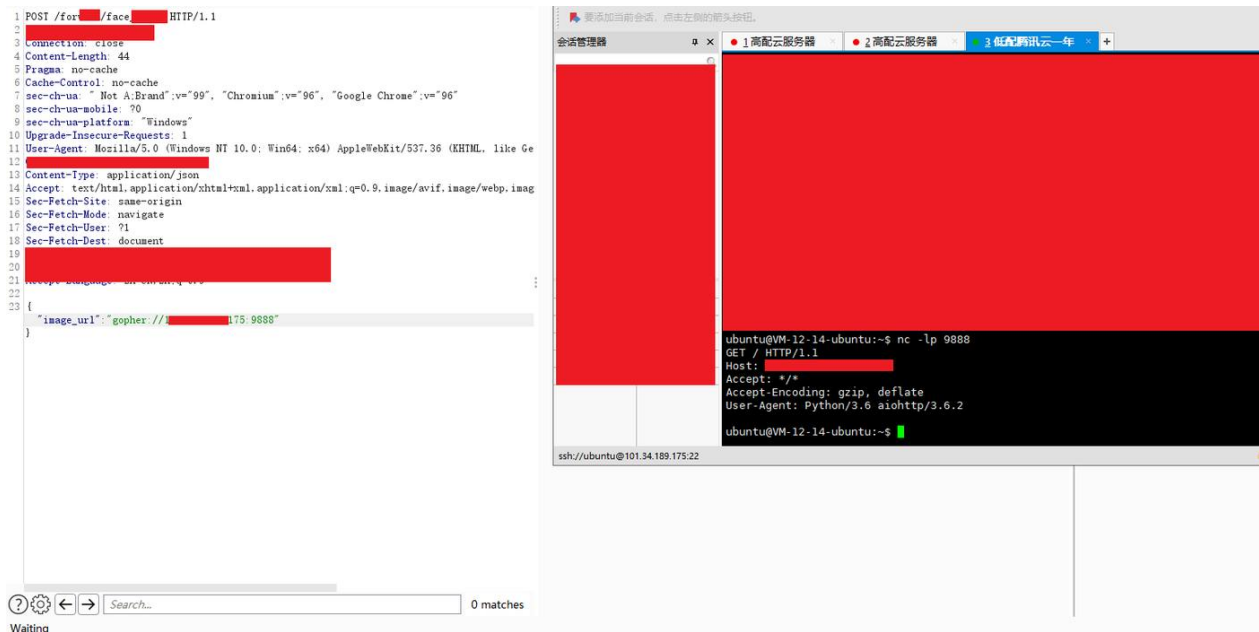
Great — keep going and FUZZ the Content-Type header (remember to turn off Burp's automatic payload encoding).

FUZZing found that `application/json` works. Simple then: build JSON data and FUZZ the JSON parameters next.

0x03 SSRF, No Brains Required

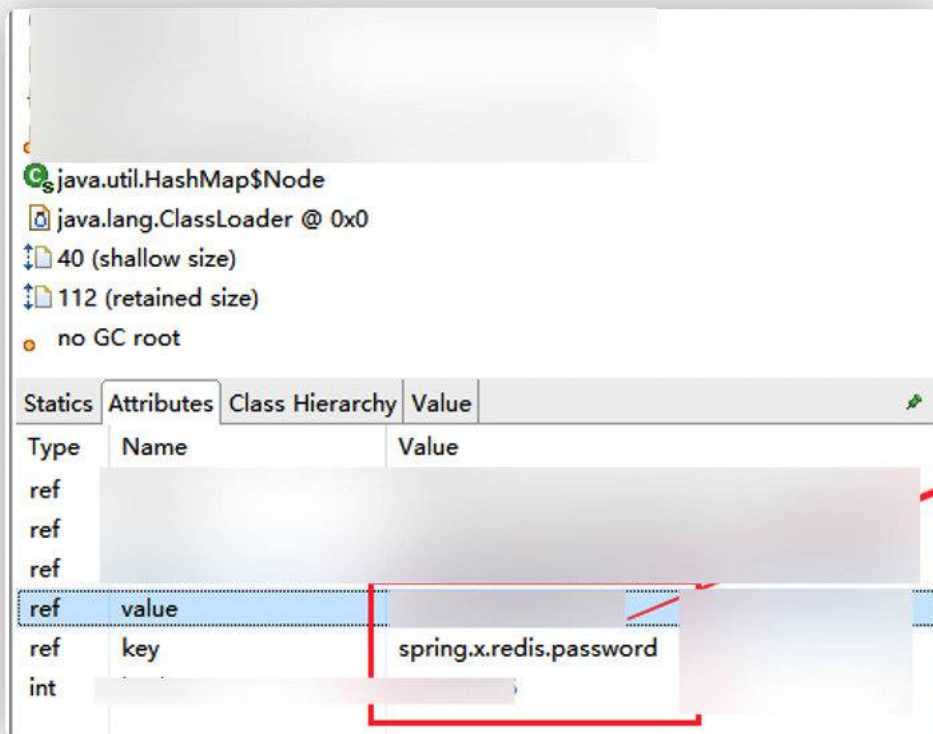
The parameter was `image_url`. Anyone with a bit of experience can tell: this parameter most likely loads a remote image.

Straight to SSRF testing.



My server received the request. After testing, the usual protocols — gopher, dict, http — all worked.

Earlier, through Spring Actuator leaks in second-level or root directories across the vendor's domains, I had downloaded heapdumps and used OQL debugging to extract Redis plaintext passwords.



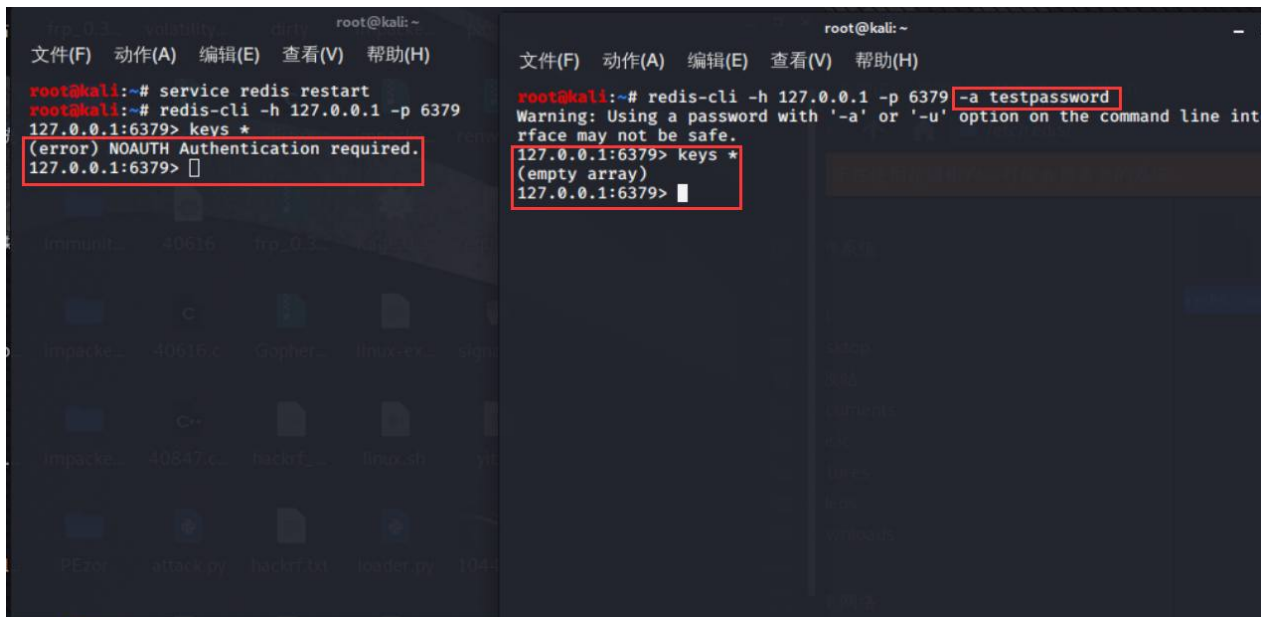
I had collected quite a few intranet Redis IPs and passwords for this vendor, and had mapped out their intranet ranges.

With this SSRF, it was entirely possible to batch password-spray the intranet Redis instances and catch reverse shells to break through the perimeter.

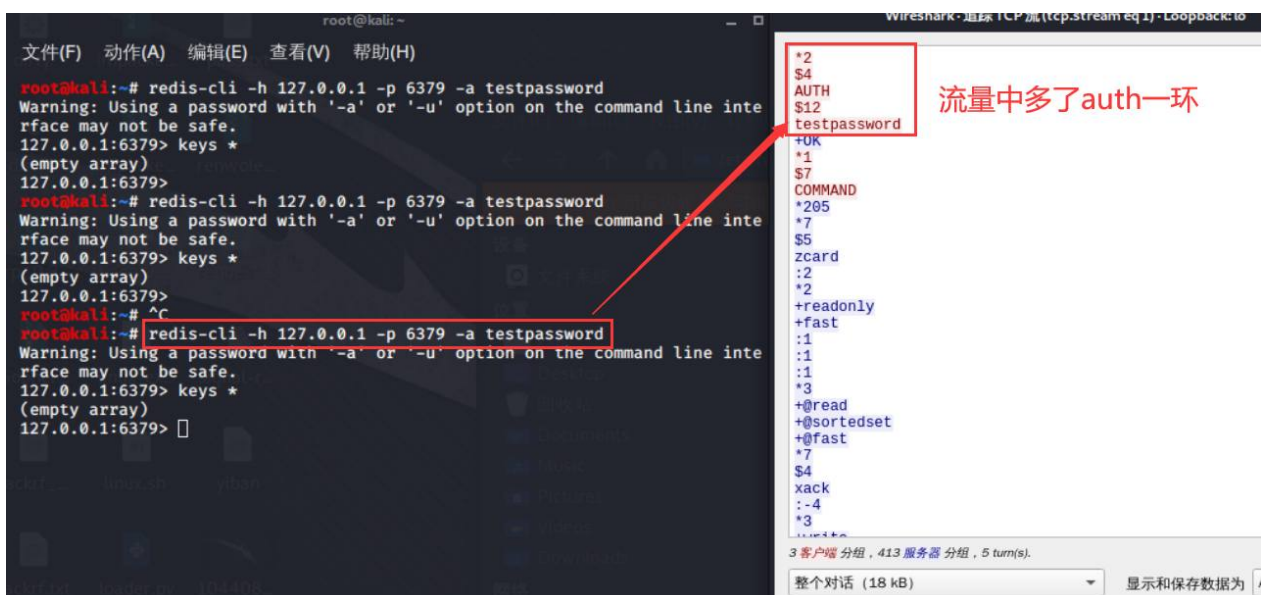
0x04 Gopher-Based Mass Redis Password Spraying and Reverse Shells on a Fragile Intranet Range

A bit of background knowledge: unlike an unauthenticated Redis, a Redis with password auth requires an extra `-a` argument to supply the password on the command line.

As shown in the figure, without the password argument no Redis command can be executed.

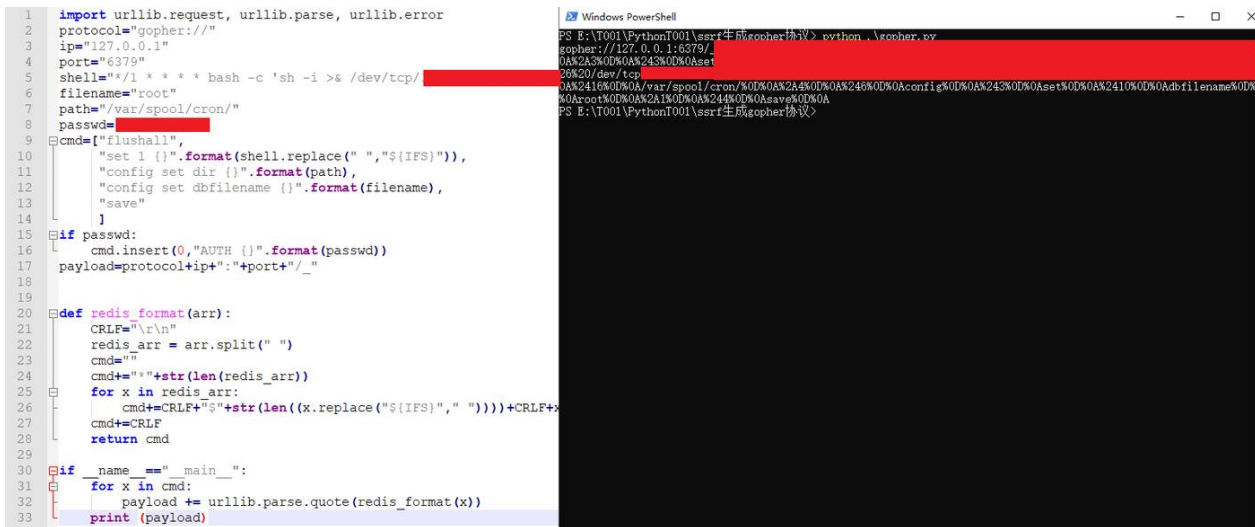


With authentication enabled, the RESP protocol traffic of the Redis session looks like this:



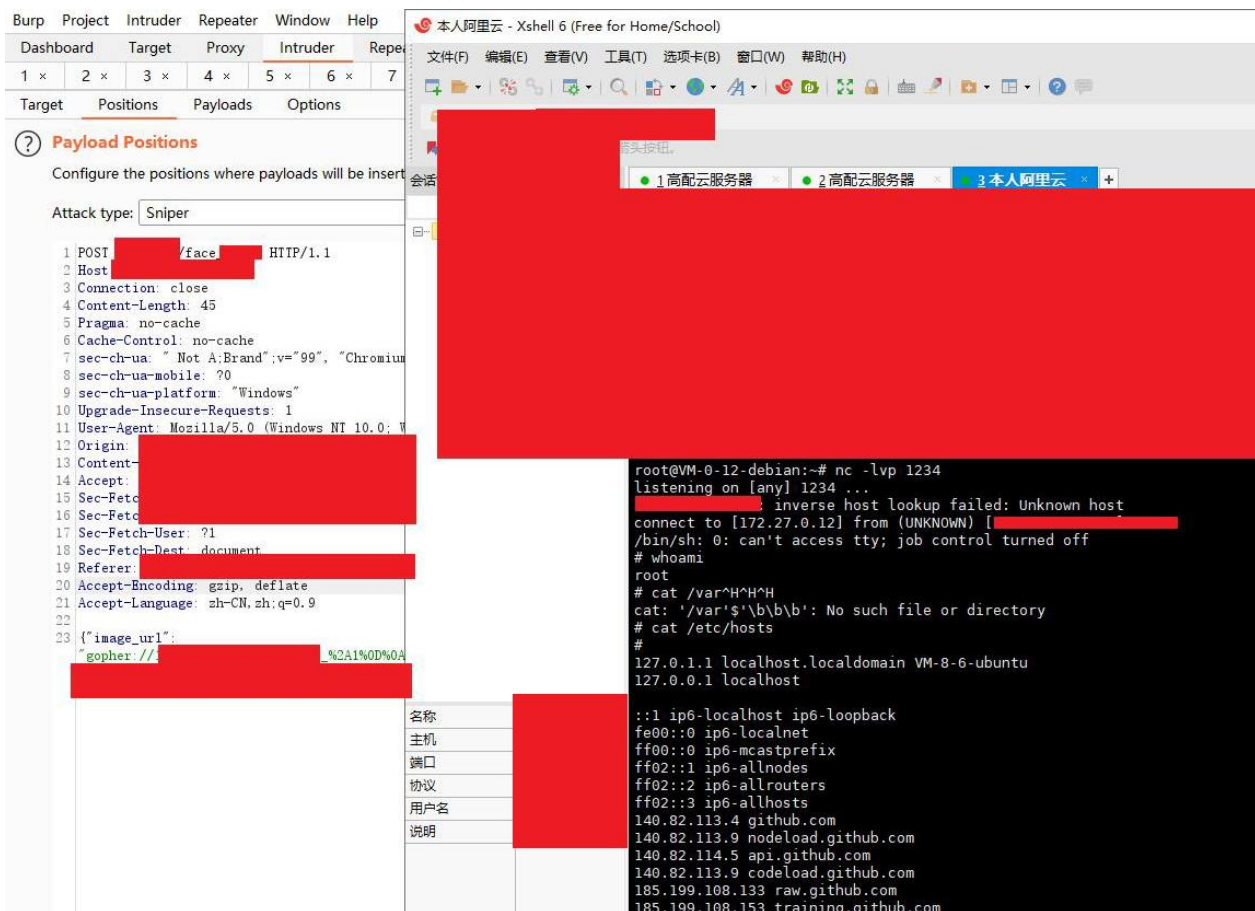
The authentication step adds an AUTH command.

I wrote a script to build the gopher payload, taking care to include this AUTH step, then did the usual: write a cron job to pop a reverse shell.



Using the SSRF point found above, combined with the intranet Redis passwords and fragile network ranges I had collected earlier,

I batch-ran the fragile intranet ranges through Intruder, password-spraying their Redis instances. The moment a spray succeeded, a cron job would be written.



Hard work paid off: in one network segment alone, more than a dozen shells came back...

The vendor's intranet Redis hosts could even reach the internet — their intranet security is honestly a mess.

```
/bin/sh: 0: can't access tty; job control turned off
# whoami
root
# ls
# ls-^H
/bin/sh: 3: ls: not found
# w
 00:05:12 up 166 days,  7:21,  0 users,  load average: 2.86, 2.77, 2.69
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
# ls -lah
total 44K
drwx-----  7 root    root    4.0K Aug 25 19:19 .
drwxr-xr-x 31 root    root    4.0K Dec 25 00:05 ..
-rw-----  1 root    root      2 May  7 2021 .bash_history
-rw-r--r--  1 ubuntu ubuntu  3.2K Jul  1 16:06 .bashrc
drwx-----  2 root    root    4.0K Aug  9 2018 .cache
drwx-----  3 root    root    4.0K Aug  9 2018 .gnupg
drwxr-xr-x  3 root    root    4.0K Aug 25 19:19 .local
drwxr-xr-x  2 root    root    4.0K May  7 2021 .pip
-rw-r--r--  1 root    root   148 Aug 17 2015 .profile
-rw-r--r--  1 root    root    73 May  7 2021 .pydistutils.cfg
drwx-----  2 root    root    4.0K May 29 2019 .ssh
-rw-----  1 root    root      0 May  7 2021 .viminfo
# cd ../
```

0x05 Afterword

I found this bug on Christmas Eve~ Consider it a Christmas present.